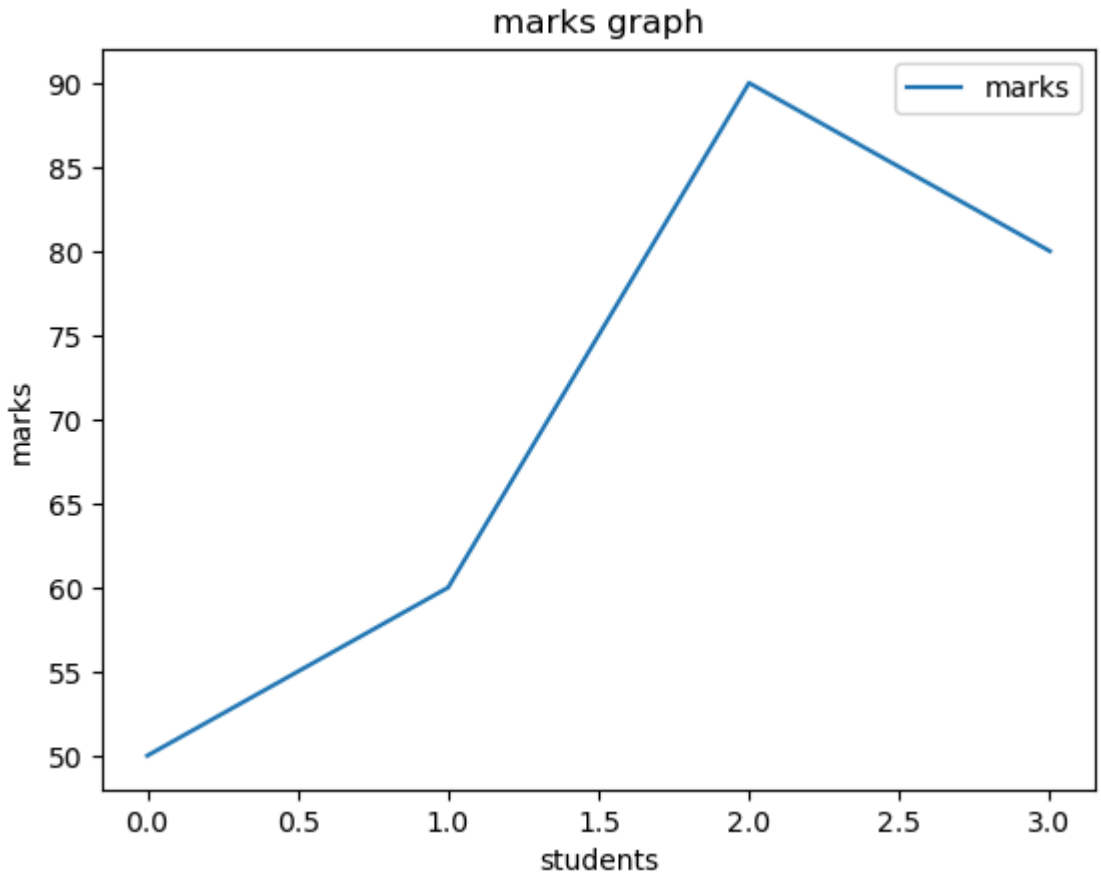
In [1]:
```
#Week-1
# Write a python program to compute Central Tendency Measures: Mean, Median, Mod

import statistics as st
import numpy as np
d=[20,10,50,3,40,3,2]
print("Mean:",st.mean(d))
print("Median:",st.median(d))
print("Mode:",st.mode(d))
print("Variance:",st.variance(d))
print("Standard deviation:",np.std(d))
print(st.stdev(d))
```

```
Mean: 18.285714285714285
Median: 10
Mode: 3
Variance: 380.23809523809524
Standard deviation: 18.053209336484713
19.49969474730554
```

In [7]:
```
# Week-2
#Study of Python Basic Libraries such as Statistics, Math, Numpy and Scipy

import statistics as st
import math as mp
import numpy as np
from scipy import stats
d=[1,2,3,4]
print("mean:",st.mean(d))
print("mode:",st.mode(d))
print("power:",mp.pow(3,2))
print("pi:",mp.pi)
print("root:",mp.sqrt(7))
print("SD:",np.std(d))
print("SD:",stats.tstd(d))
print("variance:",stats.tvar(d))
```

```
mean: 2.5
mode: 1
power: 9.0
pi: 3.141592653589793
root: 2.6457513110645907
SD: 1.118033988749895
SD: 1.2909944487358056
variance: 1.6666666666666667
```

In [5]:
```
#Week-3
#Study of Python Libraries for ML application such as Pandas and Matplotlib

import pandas as pd
import matplotlib.pyplot as plt
df=pd.DataFrame({"marks":[50,60,90,80]})
df.plot(kind="line")

plt.xlabel("students")
plt.ylabel("marks")
plt.title("marks graph")
plt.show()
```



In [21]:
```
#Week-4
#Write a Python program to implement Simple Linear Regression

import numpy as np

x=np.array([1,3,5,7,13])
y=np.array([2,4,6,7,8])
m=np.sum((x-x.mean())*(y-y.mean()))/np.sum((x-x.mean())**2)
c=y.mean()-m*x.mean()

print("m=",m)
print("c=",c)
```

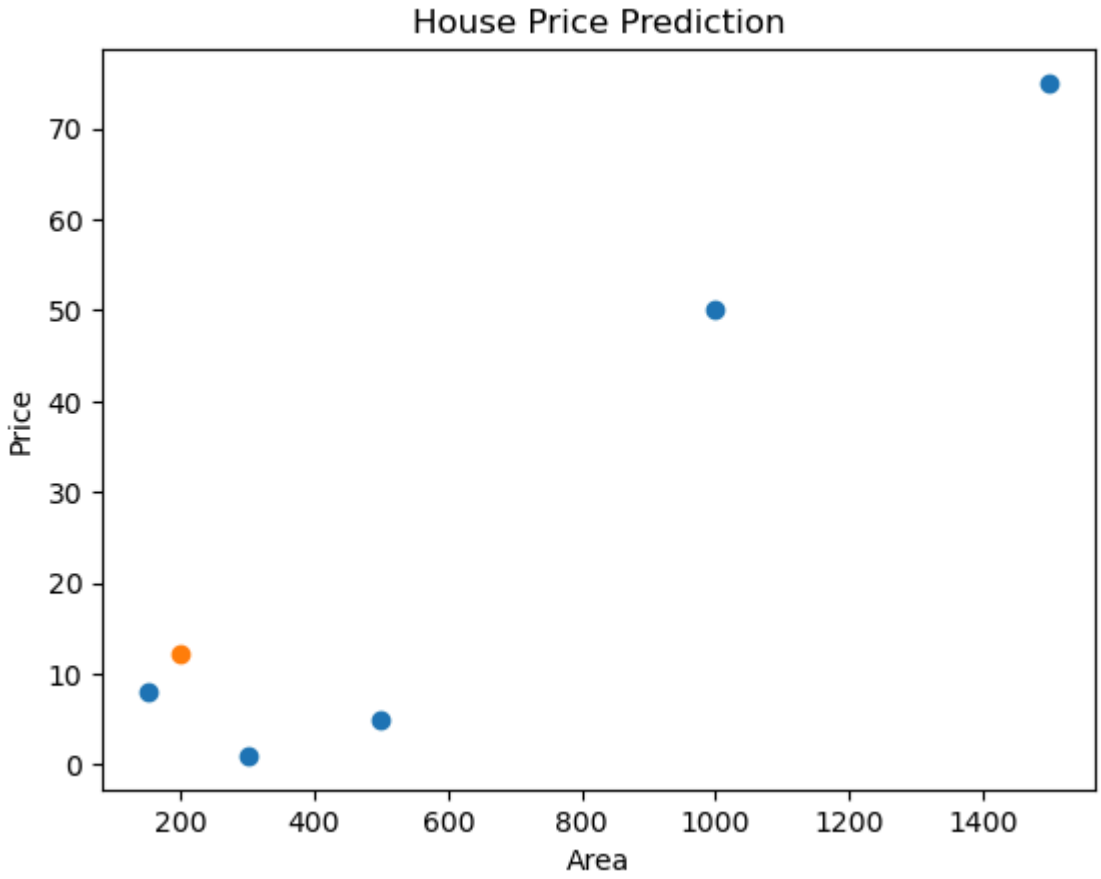
```
m= 0.47641509433962254
c= 2.6367924528301896
```

In [13]:
```
#Week-5
#Implementation of Multiple Linear Regression for House Price Prediction using s
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
data={
    'area':[1000,1500,300,500,150],
    'bedrooms':[2,3,6,8,3],
    'age':[10,5,9,2,8],
    'price':[50,75,1,5,8]
}
df=pd.DataFrame(data)
print(df)

x=df[['area','bedrooms','age']]
```

```
y=df['price']

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)

model=LinearRegression()

model.fit(x_train,y_train)

predicted_price=model.predict([[200,4,3]])
print("predicted price:",predicted_price[0])

print("intercept:",model.intercept_)
print("coefficents:",model.coef_)

plt.scatter(df['area'],df['price'])

plt.scatter(200,predicted_price)
plt.xlabel("Area")
plt.ylabel("Price")
plt.title("House Price Prediction")

plt.show()
```

```
   area  bedrooms  age  price
0  1000         2   10     50
1  1500         3    5     75
2   300         6    9      1
3   500         8    2      5
4   150         3    8      8
predicted price: 12.16406249999983
intercept: 29.94791666666646
coefficents: [ 0.04630208 -5.63802083 -1.49739583]
```

```
C:\Users\Preethi\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but LinearRegression was fitted with feature n
ames
  warnings.warn(
```



In [15]:
```
#Week-6
#Implementation of Decision tree using sklearn and its parameter tuning


from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split,GridSearchCV
from sklearn.metrics import accuracy_score
x,y=load_iris(return_X_y=True)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)
model=DecisionTreeClassifier()
model.fit(x_train,y_train)
pred=model.predict(x_test)

print("accuracy:",accuracy_score(y_test,pred))

params={"max_depth":[2,3,4,5]}
grid=GridSearchCV(DecisionTreeClassifier(),params)
grid.fit(x_train,y_train)

best_model=grid.best_estimator_
best_pred=best_model.predict(x_test)

print("tuned accuracy :",accuracy_score(y_test,best_pred))
```

```
accuracy: 1.0
tuned accuracy : 1.0
```

In [45]:
```
#Week-7
#Implementation of KNN using sklearn

import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

data={
    'area':[1000,1500,300,500,150],
    'bedrooms':[2,3,6,8,3],
    'price':[50,75,1,5,8]
}

df=pd.DataFrame(data)
print(df)
x=df[['area','bedrooms']]
y=df['price']

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)

scaler=StandardScaler()
x_train=scaler.fit_transform(x_train)
x_test=scaler.transform(x_test)

knn=KNeighborsRegressor(n_neighbors=3)
knn.fit(x_train,y_train)

y_pred=knn.predict(x_test)
print("predicted prices:",y_pred)

print("mse:",mean_squared_error(y_test,y_pred))

new_house=scaler.transform([[1400,3]])

print("predicted price:",knn.predict(new_house))
```

```
   area  bedrooms  price
0  1000         2     50
1  1500         3     75
2   300         6      1
3   500         8      5
4   150         3      8
predicted prices: [28.]
mse: 484.0
predicted price: [28.]
```

```
C:\Users\Preethi\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but StandardScaler was fitted with feature nam
es
  warnings.warn(
```

In [21]:

```
#Week-8
#Implementation of Logistic Regression using sklearn

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

x,y=load_iris(return_X_y=True)

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)

model=LogisticRegression(max_iter=200)
```

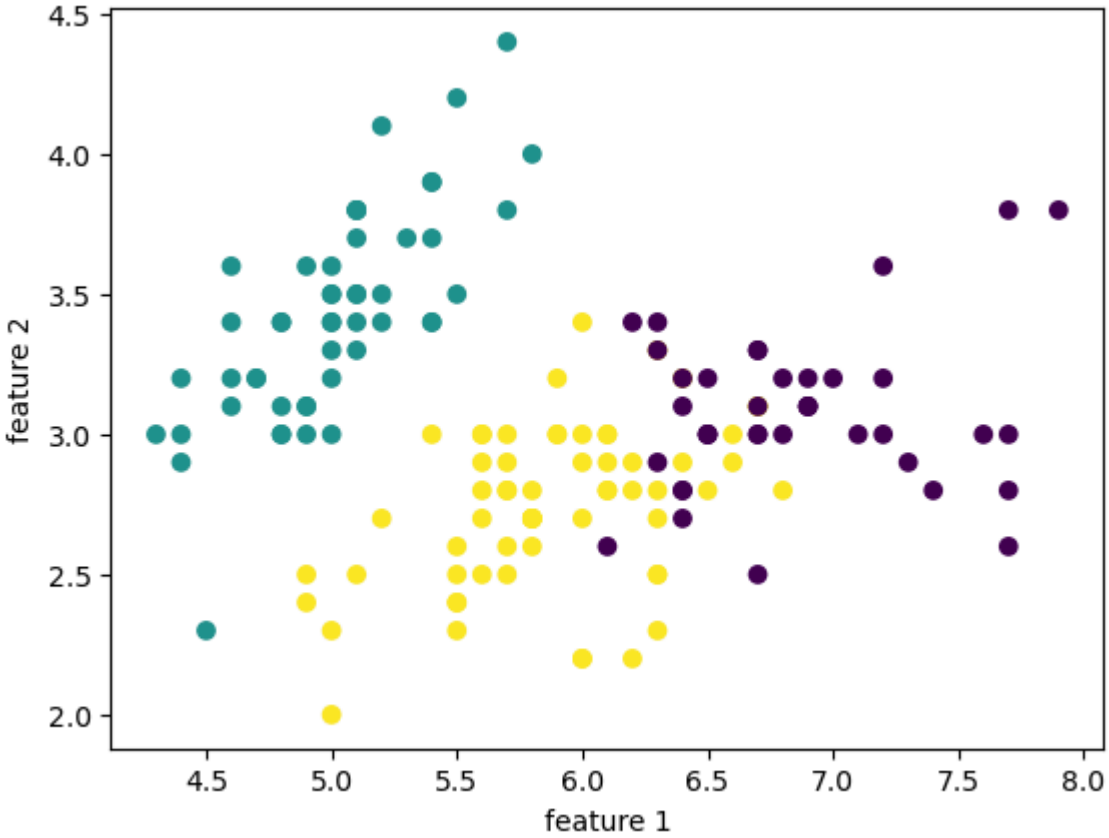
```
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
print("accuracy:",accuracy_score(y_test,y_pred))
```

```
accuracy: 1.0
```

In [55]:

```
# Week-9
# Implementation of K-Means Clustering

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

iris = load_iris()
x = iris.data

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(x)

labels = kmeans.labels_

print("cluster labels:\n", labels)
print("cluster centers:\n", kmeans.cluster_centers_)
print()

plt.scatter(x[:,0], x[:,1], c=labels)

plt.xlabel("feature 1")
plt.ylabel("feature 2")

plt.show()
```

```
C:\Users\Preethi\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: Use
rWarning: KMeans is known to have a memory leak on Windows with MKL, when there a
re less chunks than available threads. You can avoid it by setting the environmen
t variable OMP_NUM_THREADS=1.
  warnings.warn(
cluster labels:
 [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 0 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0 0 0 0 2 0 0 0 0
 0 0 2 2 0 0 0 0 2 0 2 0 2 0 0 2 2 0 0 0 0 0 2 0 0 0 0 2 0 0 0 2 0 0 0 2 0
 0 2]
cluster centers:
 [[6.85384615 3.07692308 5.71538462 2.05384615]
 [5.006      3.428      1.462      0.246     ]
 [5.88360656 2.74098361 4.38852459 1.43442623]]
```



In [ ]: